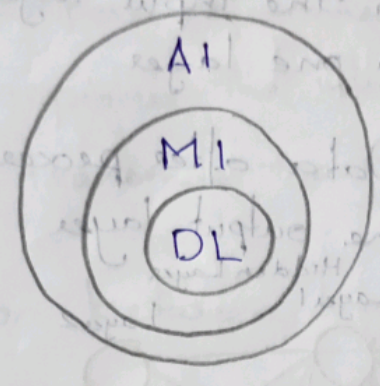
INDEX

NAME Suhana Shaju SUBJECT Deep Learning

STD. MCA S3 DIV. _____ ROLL NO. _____ SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS

Module 1



Deep Learning

A technique to perform machine learning inspired by our brains own neurons.

Biological Neurons

An ~~applied~~ artificial neuronal network which is a computational model based on structure and function of biological neural network. It is like an artificial ~~neuron~~ human neurons system for receiving, transmitting, processing info

It consists of number of neurons structured into different layers.

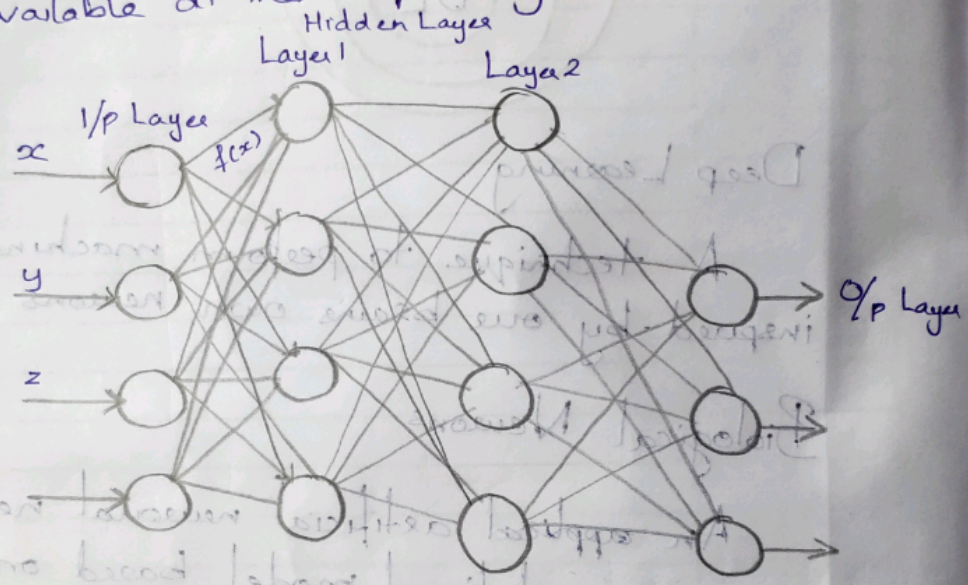
There are 3 different layers

1. Input Layer
2. Hidden Layer
3. Output Layer

Input layer : All the inputs are fed in the model through this layer.

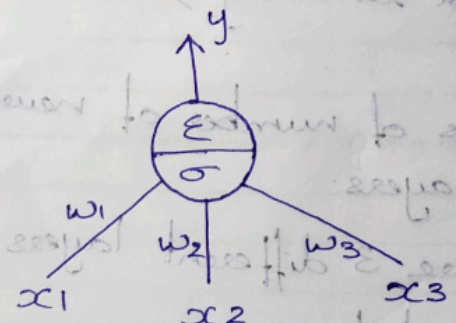
Hidden layer: Used for processing the input received from the input layer. There can be more than one layer

Output layer: Data after processing is made available at the output layer

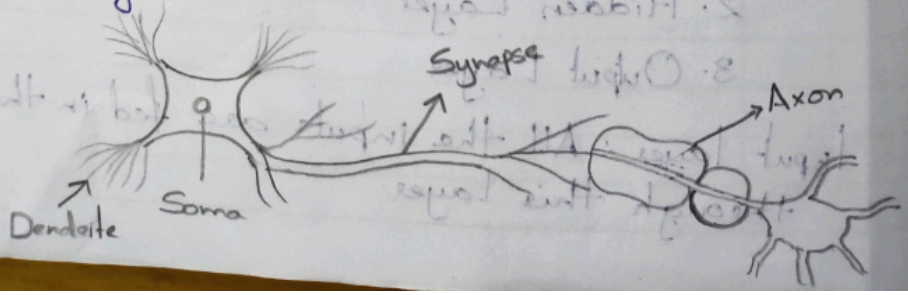


Artifical Neurons

The most fundamental unit of Deep learning



Biological Neurons



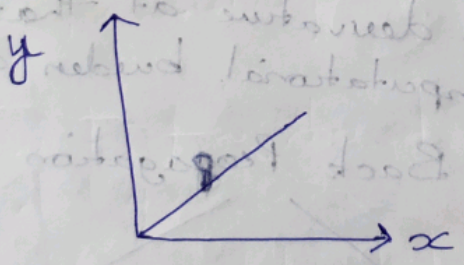
Dendrite : receive signals from other neurons
Synapse : point of connection to other neurons
Soma : process the information
Axon : transmits the o/p of this neurons

Relation between biological neuron and AN

Cell - Neuron
Dendrites - Weights / interconnection
Soma - Net i/p
Axon - O/P

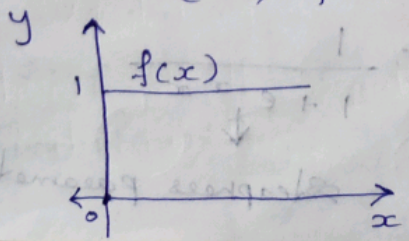
Activation Function

1. Identity Function
It is a linear function and can be defined as $f(x) = x \ \forall x$



2. Binary Step Function

$$f(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x < 0 \end{cases}$$

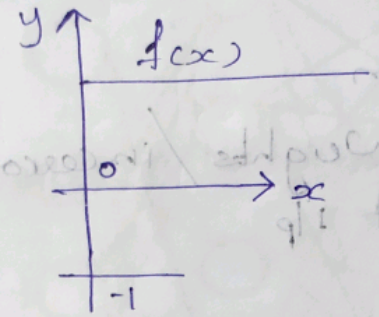


Here o/p will be binary values 0 or 1

3. Bipolar Step Function

Functions can be defined as

$$f(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ -1, & \text{if } x < 0 \end{cases}$$



4. Sigmoidal Functions

This is widely used in back propagation network because of the relationship between the value of the function at a point and the value of derivative at that point which reduces computational burden during training

Back Propagation

Binary | Bipolar

a. Binary Sigmoid function

$$f(x) = \frac{1}{1 + e^{-\sigma x}}$$

↓

Steepness parameter

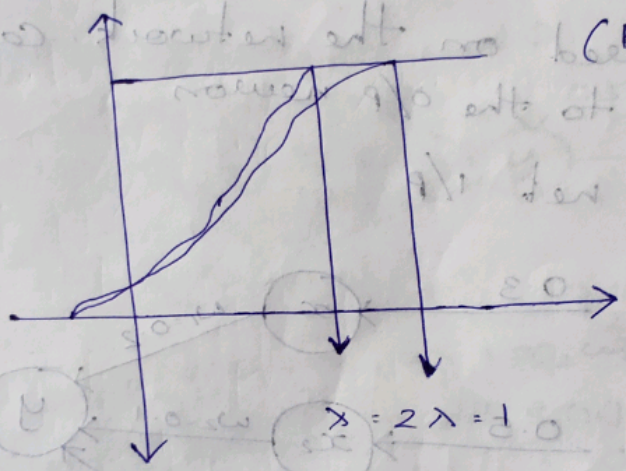
Corresponding derivative of above equation

$$f'(x) = \lambda f(x)[1 - f(x)]$$

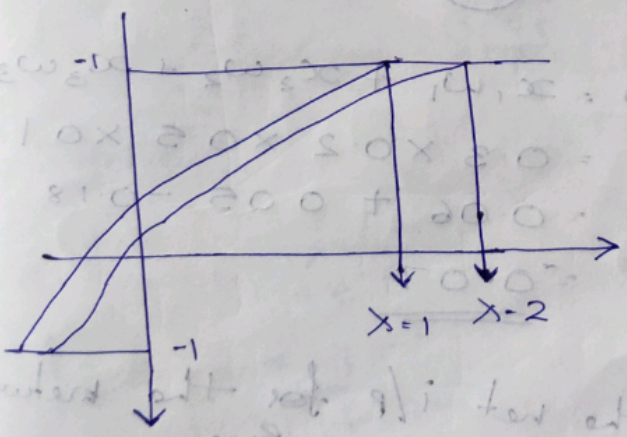
b. Bipolar Sigmoidal function

$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1$$

$$f^{-1}x = \frac{\lambda}{2}[1 + f(x)][1 - f(x)]$$

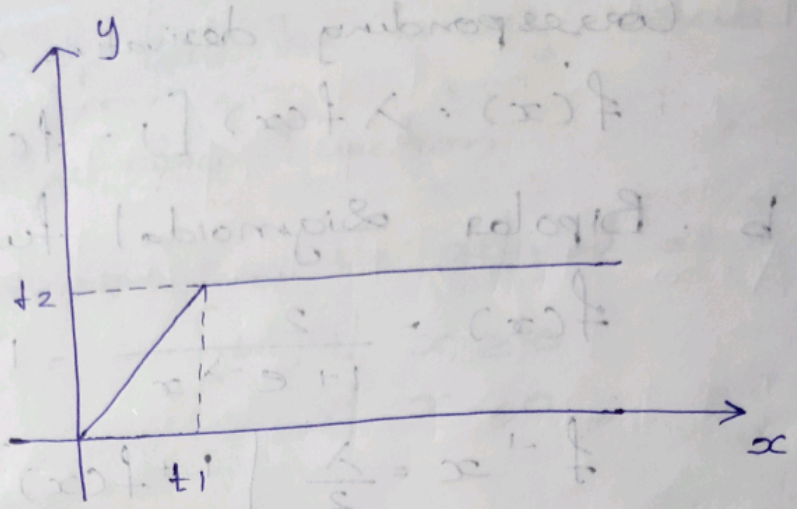


Bipolar



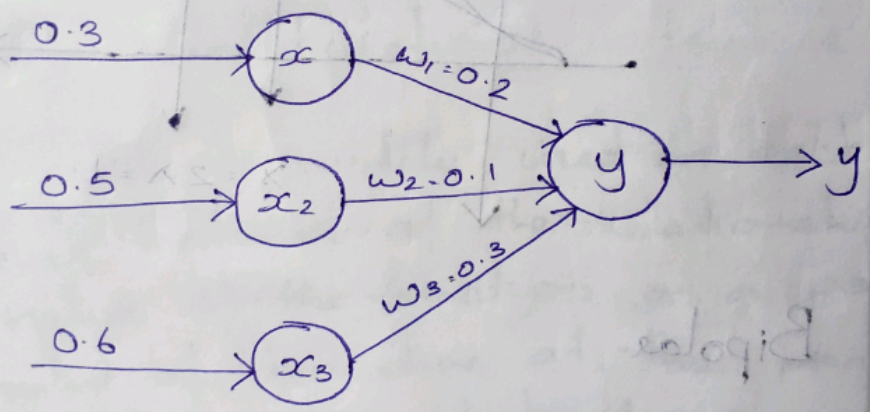
5. Ramp function

Here $f(x) = \begin{cases} 1, & \text{if } x > 1 \\ x, & \text{if } 0 \le x \le 1 \\ 0, & \text{if } x < 0 \end{cases}$



Q Based on the network calculate the net i/p to the o/p neuron

net i/p



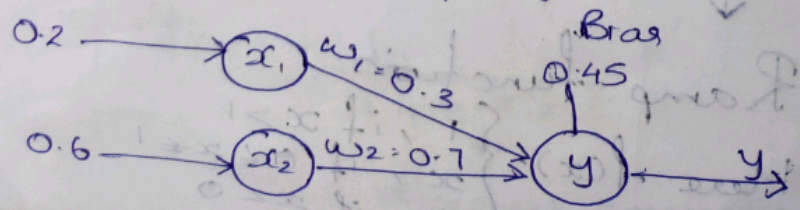
Net i/p : $x_1 w_1 + x_2 w_2 + x_3 w_3$

$= 0.3 \times 0.2 \times 0.5 \times 0.1 + 0.6 \times 0.3$

$= 0.06 + 0.05 - 0.18$

$= 0.07$

Q Calculate the net i/p for the network



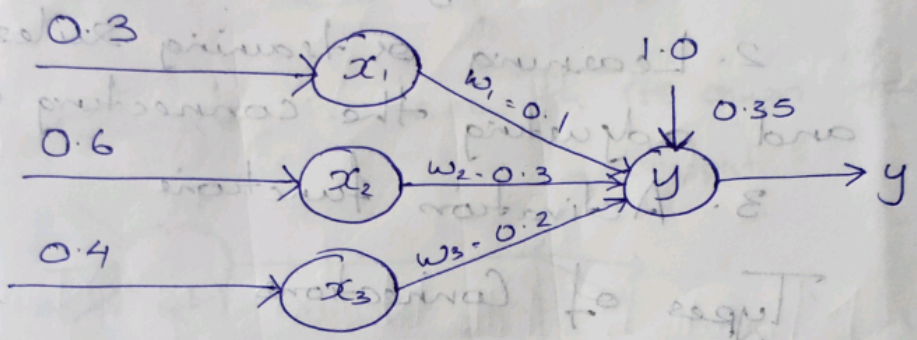
Net i/p = b $x_1 w_1 + x_2 w_2$

$= 0.45 + 0.2 \times 0.3 + 0.6 \times 0.7$

$= 0.45 + 0.06 + 0.42$

$= \underline{0.93}$

Q Obtain the o/p of the neuron using the following activation function



Net i/p : $b + x_1 w_1 + x_2 w_2 + x_3 w_3$

$= 0.35 + 0.3 * 0.1 + 0.6 \times 0.3 + 0.4 \times -0.2$

$= 0.35 + 0.03 + 0.18 - 0.08$

$= 0.35 + 0.13$

$= \underline{0.48}$

Binary

$y = \frac{1}{1 + e^{-y_{in}}}$

$= \frac{1}{1 + e^{-0.48}} = \underline{0.62}$

Bipolar

$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1$

$$= \frac{2}{1+e^{-0.48}} - 1$$

$$= 0.235$$

Basic Model of ANN

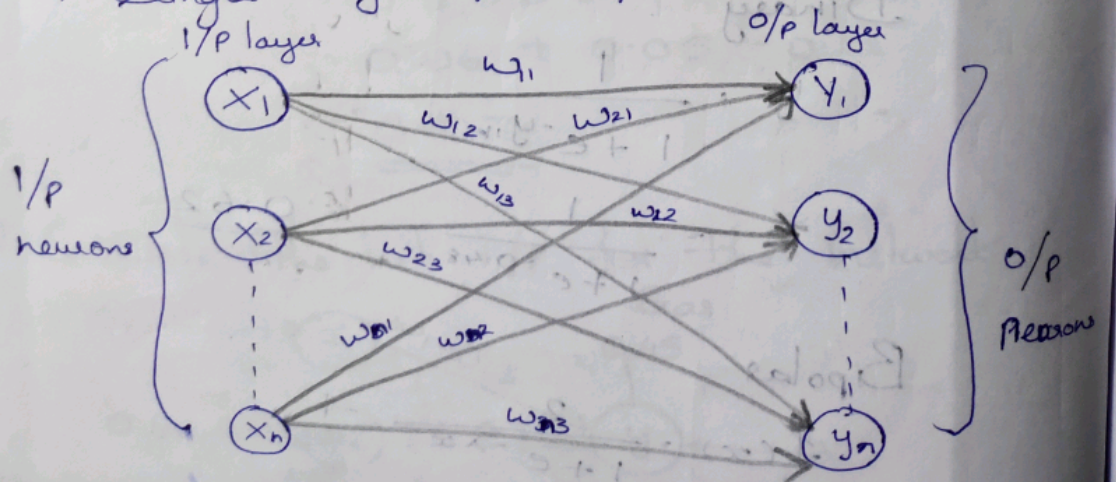
Models are specified by three basic entities

1. Models synaptic interconnections
2. Learning or training rules for updating and adjusting the connecting weight
3. Activation functions

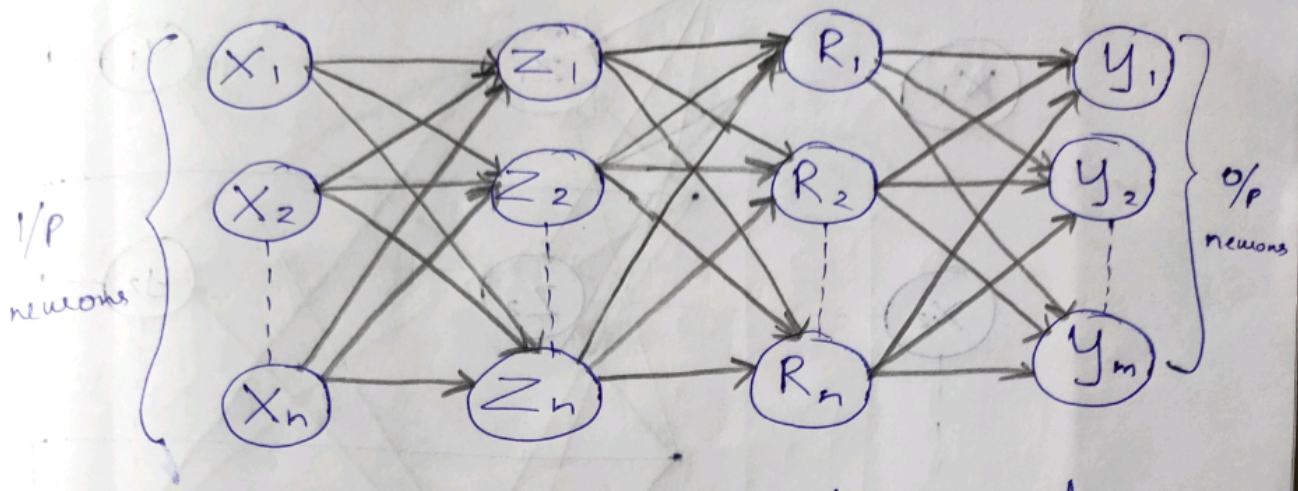
Types of Connections

1. Single layer feed forward Network
2. Multilayer feed forward Network
3. Single node with its own feedback
4. Single layer recurrent network
5. Multilayer recurrent network

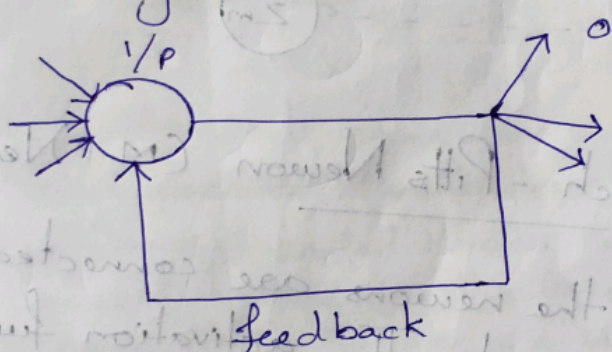
1. Single layer feed forward Network



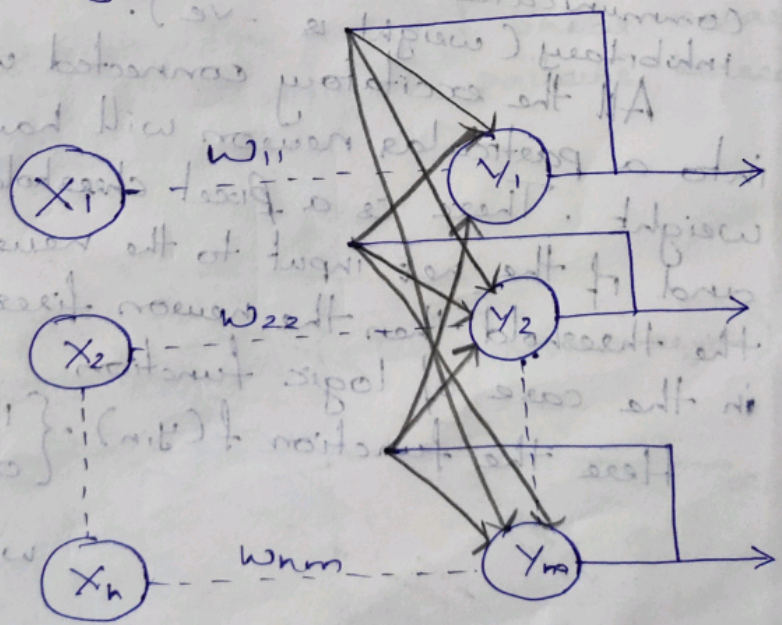
2. Multilayer feed forward Network



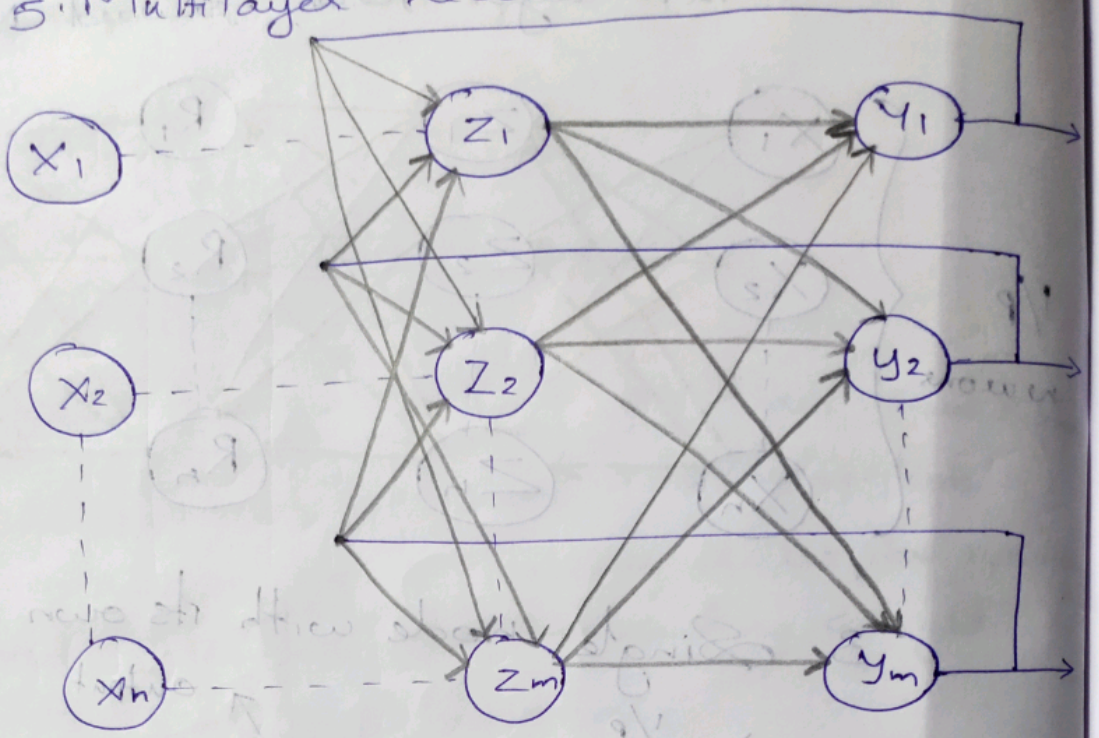
3. Single node with its own feedback



4. Single layer recurrent network



5. Multilayer recurrent network



McCulloch - Pitts Neuron (M-P Neuron)

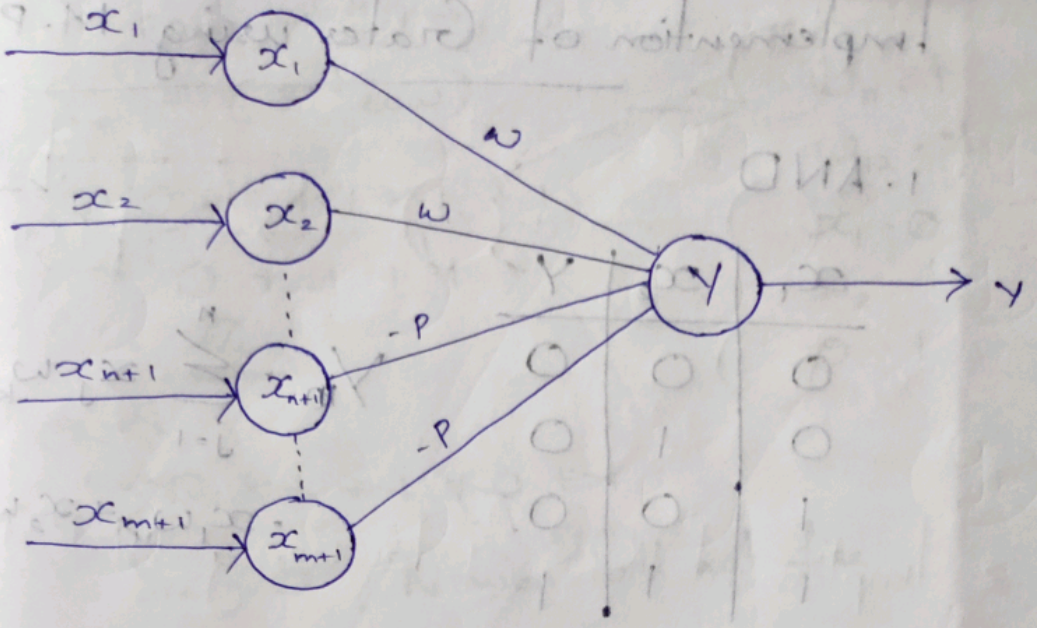
Here the neurons are connected by directed weighted graph. The activation function of MP neuron is binary, the weights associated with the communication links excitatory (weight is +ve) or inhibitory (weight is -ve).

All the excitatory connected weights entering into a particular neuron will have the same weight. There is a fixed threshold for each neuron and if the net input to the neuron is greater than the threshold then the neuron fires. It is applicable in the case of logic function.

Here the function $f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq \theta \\ 0, & \text{if } y_{in} < \theta \end{cases}$

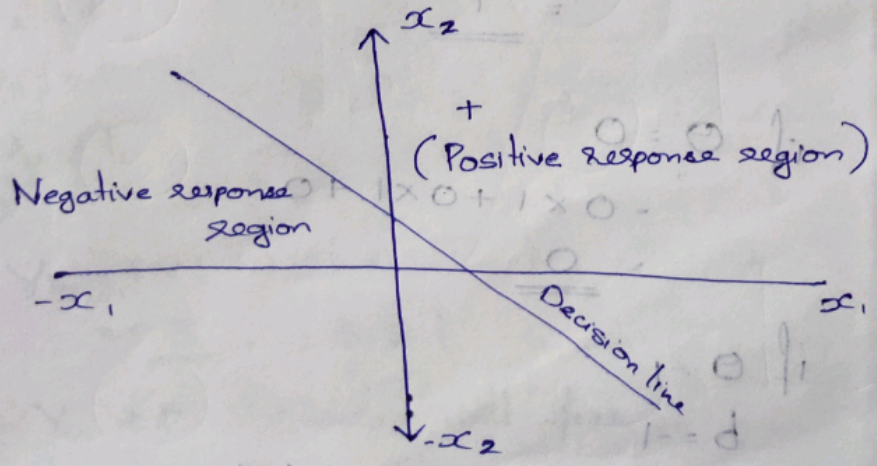
↓

will not fire case



Linear Seperability

ANN does not give an exact solution for a non linear problem. In order to solve these is another concept called linear seperablity. It is the concept where in the separation of the input space into the regions is based on whether the network response is +ve or -ve. A decision line is drawn to separate +ve and -ve responses. This decision line is called linear separable line. It is used to classify the patterns based upon their output responses.



Implemention of Grates using M-P Neuron

1. AND

x_1	x_2	Y
0	0	0
0	1	0
1	0	0
1	1	1

$$Y_{in} = \sum_{j=1}^{n} x_j w_{kj} + \theta_k$$

$$= x_1 w_{11} + x_2 w_{12} + \theta_1$$

$$f(y_{in}) = \begin{cases} 1, & y_{in} \geq \theta \\ 0, & y_{in} < \theta \end{cases}$$

Consider row 1, initialize $w_{11} = 1$

$w_{12} = 1$

$\theta_k = 1$

$$Y_{in} = \sum_{j=1}^{n} x_j w_{kj} + \theta_k$$

$$= x_1 w_{11} + x_2 w_{12} + \theta$$

$$= 0 \times 1 + 0 \times 1 + 1$$

$$= 1$$

if $\theta = 0$

$$= 0 \times 1 + 0 \times 1 + 0$$

$$= 0$$

if $\theta = 1$

$b = -1$

$$= 0 \times 1 + 0 \times 1 + 1 - 1$$

$$= 0$$

1st row satisfies

if $\theta = 1$ (2nd row) → $w_{11} = 1$, $w_{12} = -1$, $x_1 = 0$, $x_2 = 1$, $\theta = 1$

$= 0 \times 1 + 1 \times -1 + 1$

$= 0 + -1 + 1$

$= \underline{0}$

~~$\theta = 1$~~

~~$= 0 \times 1 + 1 \times 0 + 1$~~

$y_{in} < \theta$ // Neuron will not fire

if $x_1 = 1$ (3rd row)

$x_2 = 0$	0	0	0
$w_{11} = -1$	1	1	0
$w_{12} = +1$	0	0	1
$\theta = 1$	1	1	1

$y_{in} = 1 \times -1 + 0 \times 1 + 1$

$= 0$

$0 < 1$

$y_{in} < \theta$ // Neuron will not fire

if $x_1 = 1$ (4th row)

$x_2 = 1$

$w_{11} = 1$

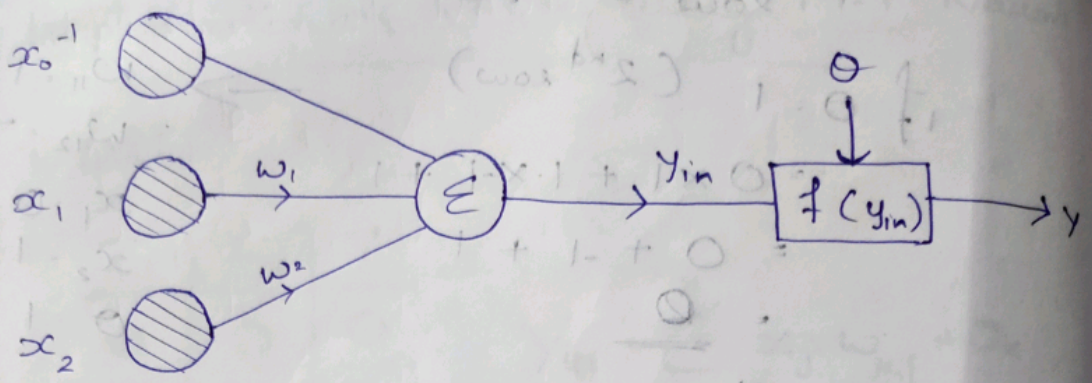
$w_{12} = 1$

$\theta = 1$

$y_{in} = 1 \times 1 + 1 \times 1 + 1$

$= \underline{3}$ $\quad 3 \geq 1$

$y_{in} \geq \theta$ // Neuron will fire



2. Implement OR gate using M-P Neuron

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

$x_1 = 0$ (1st row)
$x_2 = 0$
$w_{11} = 1$
$w_{12} = 1$
$\theta = 1$
$b = -1$

$0 \times 1 + 0 \times 1 + 1 + -1 =$
$= 0$
$\theta = -1 \quad 0 < \theta$
$y_{in} < 0$

$x_1 = 0$ (2nd row)
$x_2 = 1$

$w_{11} = 1$

$w_{12} = 1$

$\theta = 1$

$0 \times 1 + 1 \times 1 + 1$

$= \underline{2}$

$y_{in} > \theta$

Condition 1 fixed

$x_1 = 1$ (3rd row)

$x_2 = 0$

$w_{11} = 1$

$w_{12} = 1$

$\theta = 1$

$1 \times 1 + 0 \times 1 + 1 \quad : 1 + 1 = \underline{2}$

$y_{in} > 0$

$x_1 = 1$ (4th row)

$x_2 = 1$

$w_{11} = 1$

$w_{12} = 1$

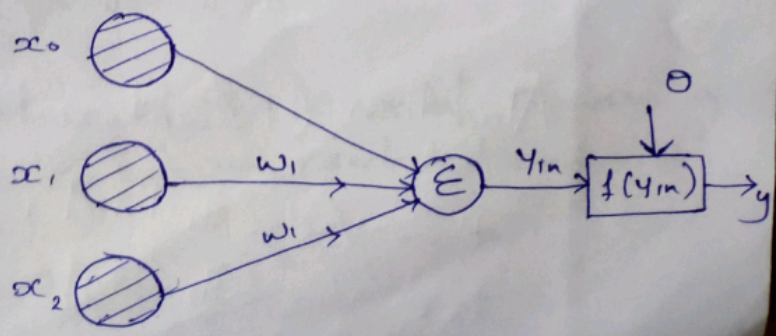
$\theta = 1$

$1 \times 1 + 1 \times 1 + 1$

$1 + 1 + 1$

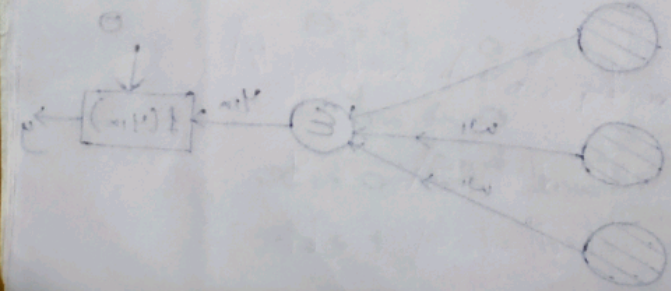
$= 3$

$y_{in} > \theta$



3 Implement of NAND Gate

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



Features of ANN

1. Activation Function
2. Network Topology - No of neurons in the model, no of layers and the way they are connected
3. Training Algorithm
   1. Perceptron Rule - It is used where the training data set is linearly separable.
   2. Delta Rule

      It is used when the training data set is not linearly separable.
4. Cost Function (scoring function / loss function / error function)

   It is the function that measures the performances of a model of the given data. Let y be the output variable $y_1, y_2, \ldots, y_n$ be the actual variable and $\hat{y_1}, \hat{y_2}, \ldots, \hat{y_n}$ be the predicted variable. Then cost functions can be written as sum of squares of differences between predicted and actual values of y.

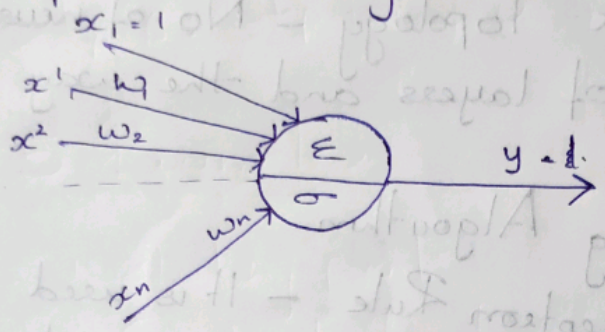
   a. SSE - $\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

   b. MSE (Mean of the sum of square of differences between predicted and actual values)

   $MSE = \frac{1}{n} \sum (y - \hat{y})^2$

Perceptron

Single perceptron can only be used to implement linearly separable function



$$\begin{cases} y = 1 & \text{if } \sum_{i=0}^{n} x_i * w_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Basic of Linear Algebra

1. Vector

$$w = [w_0, w_1, w_2, \ldots, w_n]$$

$$x = [1, x_1, x_2, \ldots, x_n]$$

$$w.x = w^T x = \sum_{i=0}^{n} x_i * w_i$$

$\|x\| \|w\|$

$\downarrow$

$\sqrt{x^2 + w^2}$

$$\cos\alpha = \frac{w^T x}{\|x\| \|w\|}$$

2. Algorithm (Perceptron learning Algorithm)

P ← i/ps with label 1

N ← i/ps with label 0

Initialize w randomly;
while ! convergence do
  Pick random $x \in P \cup N$
  if $x \in P$ and $w \cdot x < 0$ then
    $w = w + x$
  end
  if $x \in N$ and $x \cdot x \geq 0$ then
    $w = w - x$;
  end
end

Gradient Descent Algorithm

It is Iteratively Calculate, next point using gradient of current position, then scales it by a learning rate and subtract the obtained value from the current position.

Parameters

'η' (eta) - learning rate: It scales the gradient and thus control the step size when η become smaller the longer gradient descent converges

Algorithm

Step 1 : Choose a starting point (Initialization)
Step 2 : Calculate gradient at a point
Step 3 : Make a scaled step in the opposite direction to the gradient (minimize).

Step 4: Repeat step 2 and step 3 until one of the criteria is met

Step 5: Maximum no: of iteration is reached

Step 6: Step size is smaller than the tolerance

5 Parameters

1. Starting point
2. Gradient function
3. Learning rate
4. Maximum number of iteration
5. Tolerance

Algorithm

Initialize w, b;

Iterate over data;

compute $\hat{y}$

Compute $L(w, b)$

$$w_{t+1} = w_t - \eta \Delta w_t$$

$$b_{t+1} = b_t - \eta \Delta b_t$$

where $\Delta w_t = \frac{\partial L(w,b)}{\partial w}$

at $w = w_t, b = b_t$

$$\Delta b_t = \frac{\partial L(w,b)}{\partial b}$$

at $w = w_t, b = b_t$

Back Propagation Algorithm

It is an algorithm for supervised learning of ANN using gradient descent. Here the i/ps are an ANN network error function the method to calculate the gradient of error wrt to neural network weights.

Steps

1. Initially weights are assigned at random
2. The algorithm iterates through main cycle to two phases
    1. foward phase
    2. Backward phase

1. foreward Phase

Neuron are activated in the sequences from the input to output layers. Applying neuron weights and activation function along the way an output signal is produced from the finial layer

2 Backward Phase

Output signal combined with through target values. Error is propagated backward is the network to modify the corrected weights.

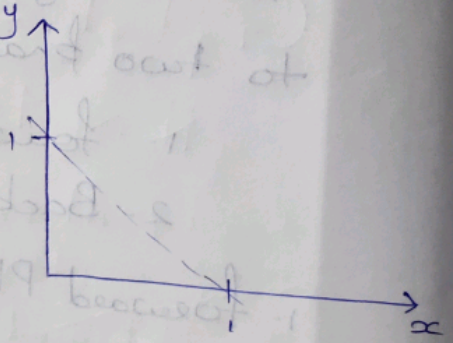
3. Technique used to determine how much a weight can be changed here gradient descent is used. technique is used.

Multilayer Perceptron (MLP)

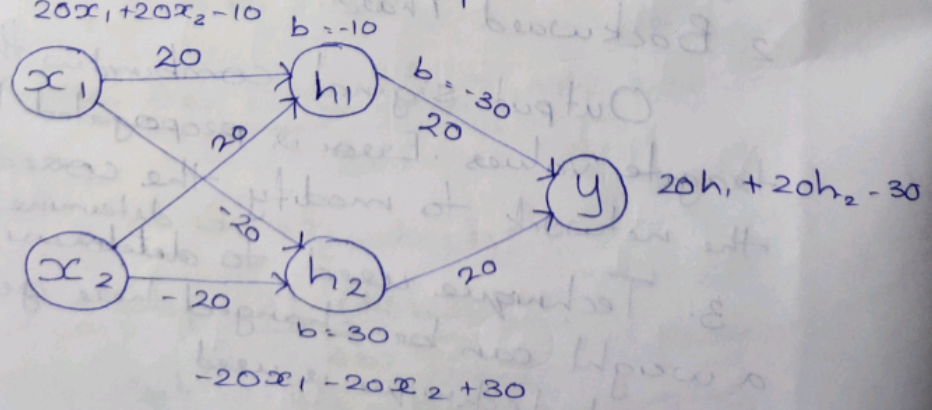
It is a feed forward ANN that generates a set of output from a set of input. Several layer of input node connected as a directed graph between input layer and output layer. Here back propogation algorithm is used.

XOR Problem using Multilayer Perceptron

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



If the input pattern are plotted according to the output it is seen that these points are linearly separable. A single neuron is not able to solve the XOR problem.



$20x_1 + 20x_2 - 10 = y$ (OR)

$(0,0)$ $0 + 0 - 10 = -10$ (0)

~~$20h_1 + 20h_2 = 30$~~

$(0,1)$ $0 + 20 \times 1 - 10 = 10$ (1)

$(1,0)$ $1 \times 20 + 0 - 10 = 10$ (1)

$(1,1)$ $20 + 20 - 10 = 30$ (1)

$-20x_1 - 20x_2 + 30$ (NAND)

$(0,0)$ $0 - 0 + 30 = 30$ (1)

$(0,1)$ $0 - 20 + 30 = 10$ (1)

$(1,0)$ $-20 - 0 + 30 = 10$ (1)

$(1,1)$ $-20 - 20 + 30 = -10$ (0)

$20h_1 + 20h_2 - 30$ (AND)

$(0,0)$ $0 + 0 - 30 = -30$ (0)

$(0,1)$ $0 + 20 - 30 = -10$ (0)

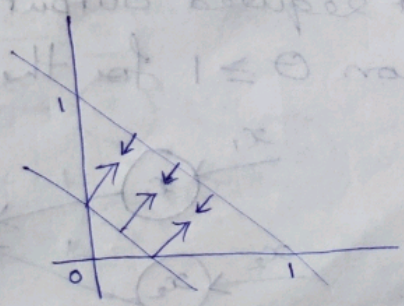
$(1,0)$ $20 + 0 - 30 = -10$ (0)

$(1,1)$ $20 + 20 - 30 = 10$ (1)

$x_1 \oplus x_2 = (x_1 \cup x_2) \cap \neg(x_1 \cap x_2)$

OR AND NAND

XOR



Implement XOR function using MP Neuron

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR function is represent as $y = x_1\bar{x}_2 + \bar{x}_1 x_2$

$$y = x_1\bar{x}_2 + \bar{x}_1 x_2$$
$$= z_1 + z_2$$
or
$$= z_1 \cup z_2$$

Compute the net inputs

$w_1 = 1, w_2 = -1$

$(0,0)$ $z_{1in} = 0\times1 + 0\times-1 = 0$

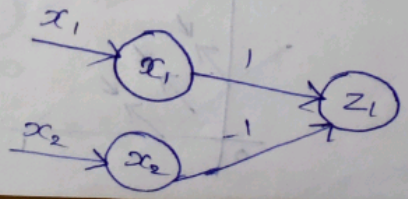
$(0,1)$ $z_{1in} = 0\times1 + 1\times-1 = -1$

$(1,0)$ $z_{1in} = 1\times1 + 0\times-1 = 1$

$(1,1)$ $z_{1in} = 1\times1 + 1\times-1 = 0$

On the basic of this input this is possible to get the ~~record~~ requred output is $w_{11} = 1$ and $w_{21} = -1$. Condition $\theta \geq 1$ for the z_1 neuron

$z_1 = x_1\bar{x}_2$



$z_2 = \bar{x}_1 x_2$ $\quad\quad$ $z_1 = x_1 \bar{x}_2$

	x_1	$\bar{x}_2$	z_1
(0,0)	0	1	0
(0,1)	1	1	1
(1,0)	0	0	0
(1,1)	1	0	0

	$\bar{x}_1$	x_2	z_1
(0,0)	1	0	0
(0,1)	1	0	0
(1,0)	0	1	1
(1,1)	0	1	0

Compute net input z_2

Assume, $w_1 = -1$ $\quad w_2 = 1$

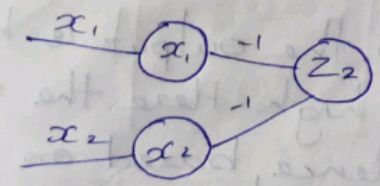
(0,0) $z_{2in} = 0 \times -1 + 0 \times 1 = 0$

(0,1) $z_{2in} = 0 \times -1 + 1 \times 1 = 1$

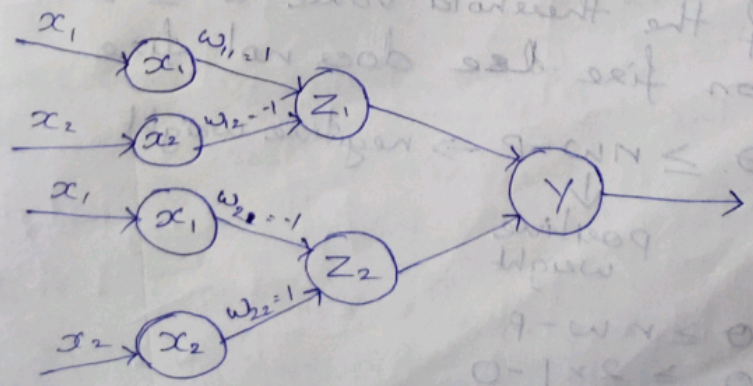
(1,0) $z_{2in} = 1 \times -1 + 0 \times 1 = -1$

(1,1) $z_{2in} = 1 \times -1 + 1 \times 1 = 0$

On the basis of the net input it is possible to get required output. Hence, $w_{21} = -1$, $w_{22} = 1$



The find neural network



Implement AND function Using MP Neurons

Assume $w_1 = 1$ $w_2 = 1$

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Assume $w_1 = 1$ $w_2 = 1$, net input calculation

$$y_{in} = x_1 w_1 + x_2 w_2$$

$(0,0) = 0 \times 1 + 0 \times 1 = 0$

$(0,1) = 0 \times 1 + 1 \times 1 = 1$

$(1,0) = 1 \times 1 + 0 \times 1 = 1$

$(1,1) = 1 \times 1 + 1 \times 1 = 2$

for AND function the output is high if both the inputs are high. Here the net input is calculated as 2. Hence based on this net input threshold value is set $\theta = 2$.

If the threshold value is ≥ 2 then the neuron fire does not fire

$\theta \geq nw - p \rightarrow$ negitive weight

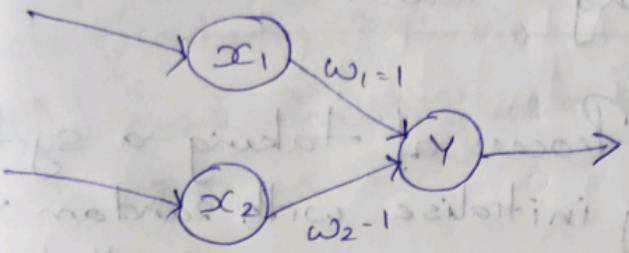
↓ positive weight

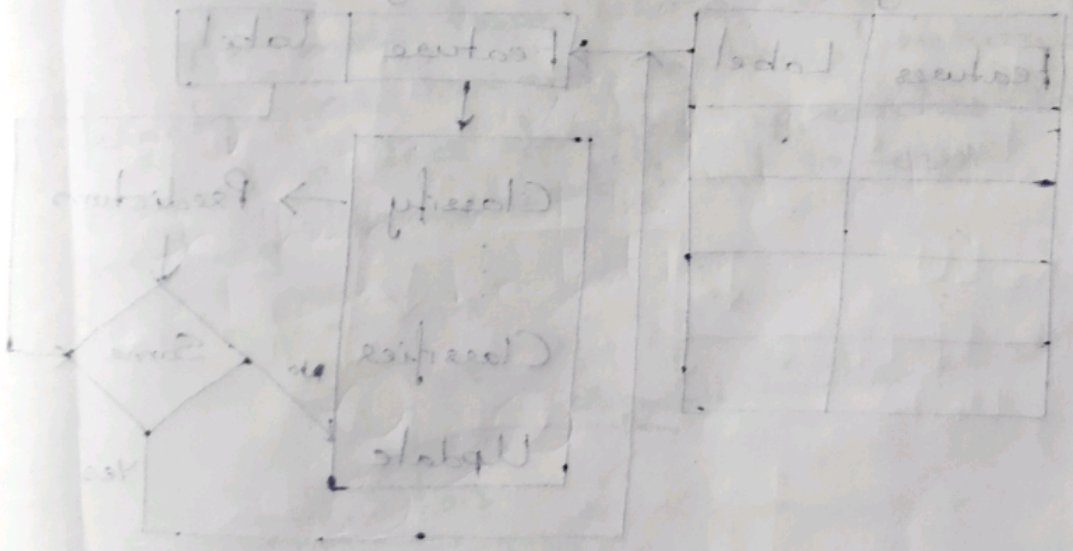
$\theta \geq nw - p$

$\theta \geq 2 \times 1 - 0$

$\theta \geq 2$

Output of neuron $y = \begin{cases} 1 & \text{if } y_{in} \geq 2 \\ 0 & \text{if } y_{in} < 2 \end{cases}$

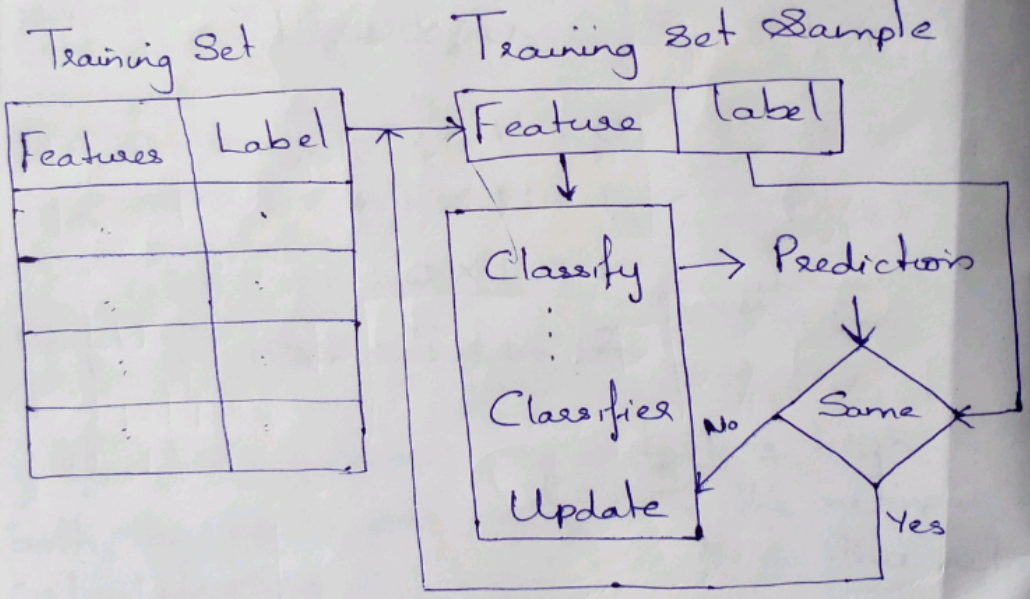




Module 2

Training Neural Networks

Training - Process of taking a system that is being initialise with random values and gradually improving. So that it is tuned to the data we want to understand. Collection of all the sample along with these labels is called training set.



Working of Training Process

1. The weight of the network are initialise to small random values then the network perform a series of training step
2. At each training set one batch of images is passed to the network and the errors are

back propogated to updated the weights.

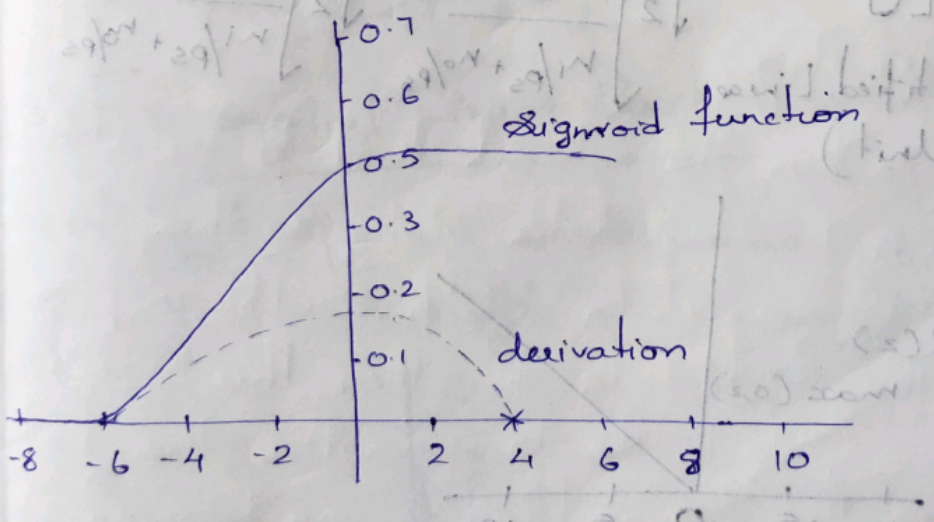
3. The larger the batch size the more stable the geadient calculation but slower each training step.

. At each training get batch size between 32 to 256

Training Deep Neural Networks

Challenges Facing Vanishing Geadient Problem

As more layer using certain activation function are added to neural network the geadient of loss function approaches to zero making the network hard to train



Solutions

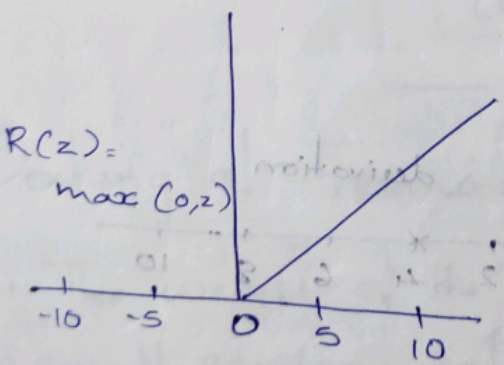
1. Exploading geadient : Aix geadient is the direction and magnitude calculated during the training of neural network that is used to update the

network weight in the right direction and by the right amount.

2. Proper weight Initialization
Saturated Activation Function
1. Xavier Initialization

~~Logistic~~

	Uniform Distribution	Normal Distribution
1 Logistic function	$\sqrt{\frac{6}{n_{i/ps} + n_{o/ps}}}$	$\sqrt{\frac{2}{n_{i/ps} + n_{o/ps}}}$
2 Hyperbolic tangent	$4\sqrt{\frac{6}{n_{i/ps} + n_{o/ps}}}$	$4\sqrt{\frac{2}{n_{i/ps} + n_{o/ps}}}$
3 ReLU (Rectified Linear Unit)	$\sqrt{2}\sqrt{\frac{6}{n_{i/ps} + n_{o/ps}}}$	$\sqrt{2}\sqrt{\frac{2}{n_{i/ps} + n_{o/ps}}}$



Using Nonsaturated Activation Function

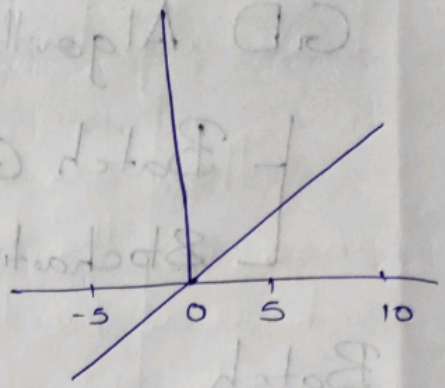
1. Leaky ReLu

$\max(\alpha z, z)$

$\alpha = 0.01$

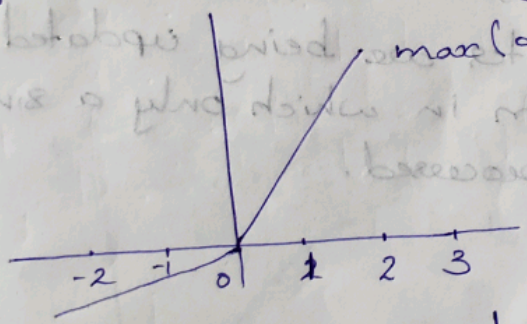
hyper parameter

$\alpha = 0.2$



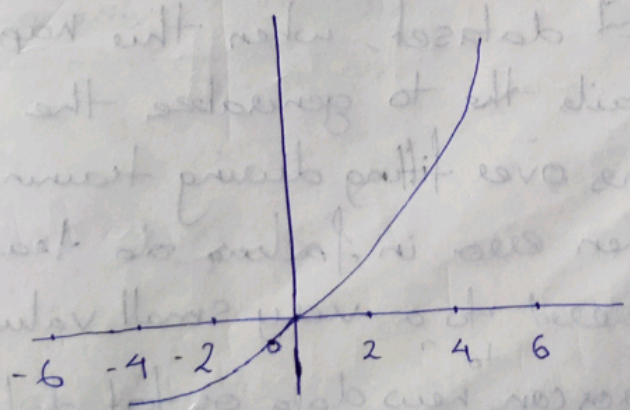
2) Randomized Leaky ReLU

α is picked randomly in a given range during training and it is fixed to an average value during testing



3) Exponential Linear Unit (ELU)

$\alpha = 1$



GD Algorithms / Types

- Batch Gradient Descent
- Stochastic GD

Batch

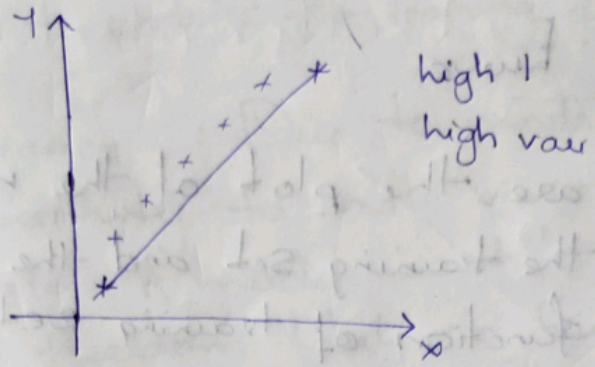
Types of GD which process all the training example

Stochastic

That process training example per iteration. Hence the parameters are being updated even after one iteration in which only a single example has been processed.

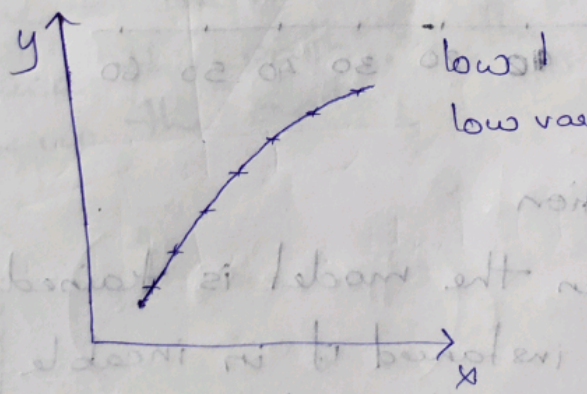
OverFitting

When the network tried to learn too much training data along with a noise from the training data which results in poor perform on the test dataset, when this happen the network fails the to generalize the feature of the patterns over fitting during training can be spotted when error in training data can be decreased to a very small value. But the error can, to new data or test data increase to a large value



Under fitting

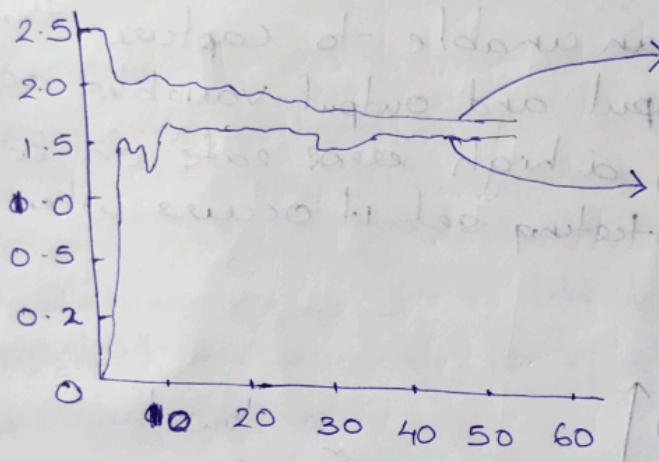
This is the scenario where the data model is unable to capture the relationship between input and output variables accurately, generating a high error rate on both training set and testing set it occurs when it is model to simple



Regression Model

Learning Curve

These are the plot of the models performance on the training set and the validation set as a function of training set size. To generate the plot simple train the model several times on different size on subset of the training set.



Validation

When the model is trained on very few training instances it is incable of generalizing properly. So the validation error is quite big.

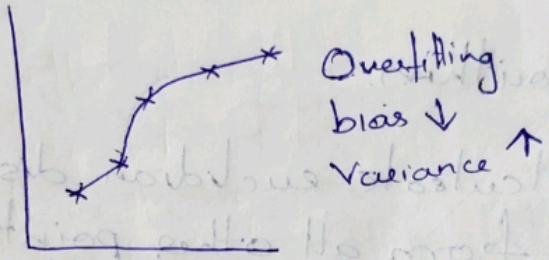
Bias / Variance Teade off

Bias : Generalistion error is due to wrong assumptions

eg:- Assuming the data is linear when it is actually Quadratic a high bais model is most

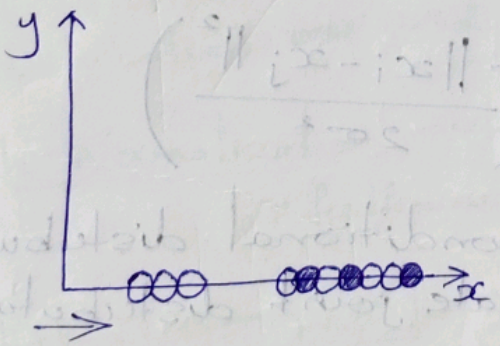
likely to underfit the training data.

Variance : Due to model excessive sensitivity to small variation on the training data. A model with many degree of freedom likely to have high variance This due to overfitting the training data.



Irreducible Error

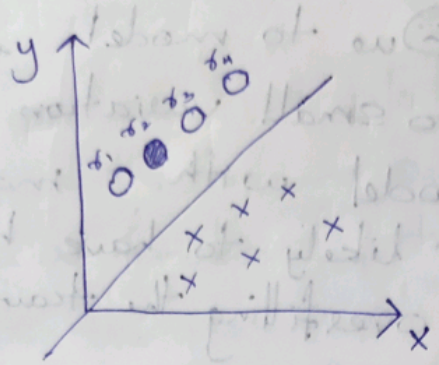
Due to the noise of data itself only way to reduce this path of the error is due to clean up the data.



T-SNE (t-distributed Stochastic Neighbour Embedding)

It is a method of dimensionality reduction mainly for visualization of data in 2D or 3D ways. This method can find non-linear connection used for

on the data and highly popular



Algorithm

1. Calculate euclidian distance of each point from all other points. Take this distance and transforming them into conditional probability then calculate how similar every point in the data. That is how lightly they are to be neighbours conditional probability given x_j given x_i is represend by a Guassian centred at x_i take with a standed division of σ

$$P_{j/i} = \frac{\varepsilon}{\exp\left(\frac{-\|x_i - x_j\|^2}{2\sigma^2}\right)}$$

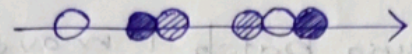
From the conditional distribution we need to calculate joint distribution

$$P_{j,i} = \frac{P_{j/i} + P_{i/j}}{2n}$$

2. Creating data in a low dimension

Create a low dimension space

Apply joint probability distribution point.



Applying joint probability distribution using t-distribution

$$q_{ij} = 1 + (\|y_i - y_j\|^2)^{-1}$$

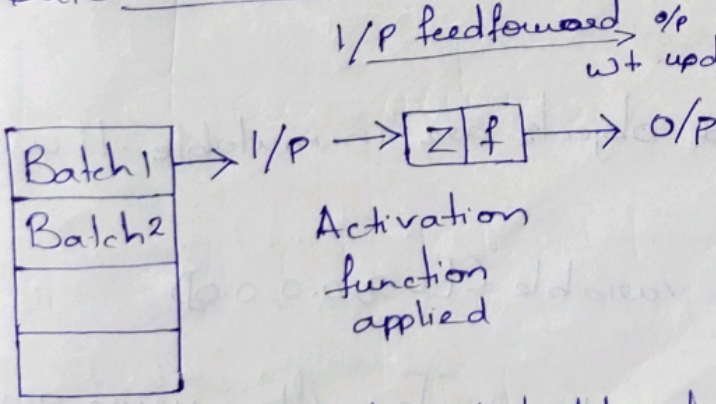
3. Apply Kullback - Leiber Divergence (KL)

It is a measure of how much 2 distribution are different from one another. Let P and Q are the distribution on the probability space X and KL divergence.

$$D_{KL}(P \| Q) = \sum_{x \in X} \log\left(\frac{P(x)}{Q(x)}\right)$$

When the distribution are similar value of KL divergence is smaller reaching zero when the distribution are identical

Batch Normalization



- Apply distribution to hidden layer
- Processing 1 batch is called epoch

epoch ↓ → o mean
and get mixed o/p ↑ variance

After processing batch 1 move to batch 2

DS for processing Batch Normalization

Python

tensorflow - It consist of graph, node, shape
⇒ No. of Dimensions

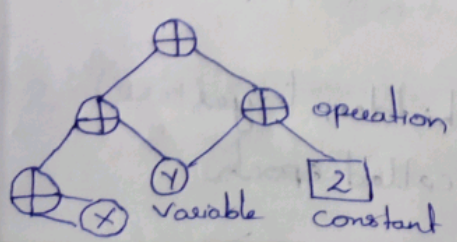
TensorFlow is an end-to-end platform (1D, 2D) for machine learning. multidimensional array based numeric computation and is represented as tf.Tensor objects

```
import tensorflow as tf
x = tf.constant([[[1., 2., 3.],
                  [4., 5., 6.]]])
print(x)
print(x.shape)
print(x.dtype)
```

Variable

tf.Tensor objects are immutable. It uses tf.Variable

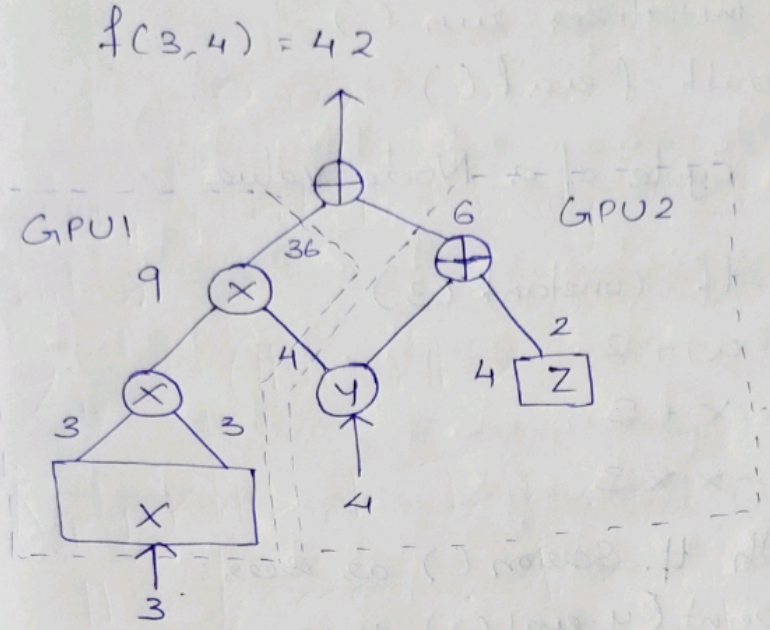
```
Var = tf.Variable([0.0, 0.0, 0.0])
```



Tensorflow using graph

$f(x,y) = x^2y + y + 2$

Graph can be break to several chunks



Create First Graph and Running it in session

```
import tensorflow as tf
x = tf.variable(3, name = 'x')
y = tf.variable(4, name = 'y')
f = x*x*y + y + 2
```

Graph can be run by calling session ie, tf.Session()

```
Sess = tf.Session()
Sess.run(x.initializer)
Sess = run(y.initializer)
result = Sess.run(f)
print(result)
 42
Sess.Close()
```

To repeat Sess.run() all the time is a bit Cumbersome, but there is a better way

```
with tf.session() as Sess:
```

```
x.initializer.run()
y.initializer.run()
result = f.eval()
```

Life cycle of a Node value

```
w = tf.constant(3)
x = w + 2
y = x + 5
z = x * 3

with tf.Session() as sess:
    print(y.eval()) #10
    print(z.eval()) #15
```

Training a DNN using Plain TensorFlow

Placeholder is to represent training data and targets

```
X = tf.placeholder(tf.float32, shape=(None,
                    n_input), name="X")
y = tf.placeholder(tf.int64, shape=(None),
                    name="y")
```

Hidden Layer

```
with tf.name_scope("dnn"):
    hidden1 = neuron_layer(X, n_hidden1, "hidden1",
                           activation="relu")
    hidden2 = neuron_layer(hidden1, n_hidden2,
                           "hidden2", activation="relu")
    logits = neuron_layer(hidden2, n_outputs, "output");
```

- fully_connected() function creates fully connected layer, all input are connected to all neurons in the layer

- Take care of creating weights and biases variables

from tensorflow.contrib.layers import fully_connected

```
with tf.name_scope("dnn"):
    hidden1 = fully_connected(X, n_hidden1,
                              scope="hidden1")
    hidden2 = fully_connected(hidden1, n_hidden2,
                              scope="hidden2")
    logits = fully_connected(hidden2, n_outputs,
                             scope="outputs",
                             activation_fn=None)
```

- last step in construction phase is specify how to evaluate the model.

Batch normalization with TensorFlow

- batch_normalization function that simply centers and normalizes the inputs, but must compute mean and std deviation and pass them as parameters to this function.

- Calculate the mean and std deviation of each of its input channels across the batch and normalizes by subtracting

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

$$\hat{x_i} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

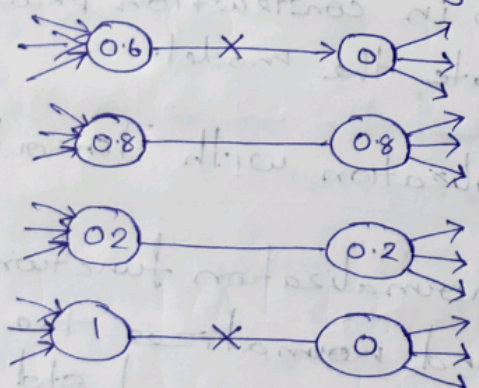
$$y_k = \gamma x_i + \beta = BN_g(x_i)$$

Parameters used in batch normalization

- Scale (gamma) and shift (beta)
- Trainable parameters
- moving average and std deviation need to be calculated for each channel

Dropout Layer

Algorithm perform well on training dataset but not the test dataset, we say that it is suffering from overfitting



Dense layer Dropout layer

Tensor Ranks, Shapes and Types

Unit of dimensionality known as rank

Introduction to keras

- High level neural network api runs on top of tensor flow

- end-to-end open source machine learning platform

```
from keras.datasets import mnist
from keras.models import sequential,
                         load_model
from keras.layers.core import Dense,
                    Dropout Activation
```

Compiling the Model

The compilation is perform using one single method called compile

Preparing Data

Before we feed the data to our network, it must be converted into the format required by the network. This is called preparing data for the network.

Normalizing Data

To normalize the data, we represent it as float type and divide it by 255

```
X_train = X_train
```

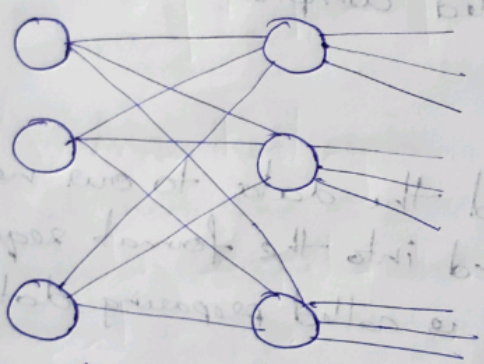
Creating Deep Learning Model

One neural network model will consist of a linear stack of layers. To define such a model, we call the sequential

```
model.sequential()
```

Input Layer

```
model.add(Dense(512, input_shape=(784,))
```




```
model.add(Activation('relu'))
model.add(Dropout(0.2))
```

Module 3

Convolution Neural Network

It is a type of ANN used in image recognition and processing that is specifically designed to process data.

Suppose image size, n, filter size f and output n-f+1

eg:-

a	b	c	d
e	f	g	h
i	j	k	l
m	n	o	p

×

w	x
y	z

$aw+bx+ey+fz$ $bw+cx+fy+gz$

$cw+dx+gy+hz$

$ew+fx+iy+jz$ $fw+gx+jy+kz$ $gw+hx+ky+lz$

$iw+jx+my+nz$ $jw+kx+ny+oz$ $kw+lx+oy+pz$

Padding in CNN

$n+2p-f+1$

0					
0	a	b	c	d	
0	e	f	g	h	
0	i	j	k	l	
0	m	n	o	p	
0					

w	x
y	z

$4+2-2+1$

Max Pooling

It is a pooling operation that select the maximum element from the region of feature map covered by the filter. Thus the output after max pooling would be a feature map containing the most prominent feature of the previous feature map

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool →

filter (2×2)
stride (2,2)

9	7
8	6

Average Pooling

To computes the average of the elements present in the region of feature map covered by the filter. Thus while max pooling gives the most prominent feature in a particular patteen of the feature map, average pooling gives the average of feature present on the patteen.

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

→

4.25	4.25
4.25	3.5

Fully connected layers

Consider the dense network of neurons every input is connected to every o/p. Then it is called fully connected layer.